METHOD BRIEF · TECHNICAL

Verified migration of a legacy stored-procedure estate

Replay-based behavioural equivalence: what it proves, what it cannot, and the failure modes that decide whether it is safe to run unattended.

ABSTRACT

Legacy modernisation is bottlenecked not by code generation but by the **oracle problem** — the absence of any procedure that decides whether a candidate replacement is acceptable. This brief sets out a method that sidesteps it by substituting a weaker but decidable criterion: **observational equivalence to the incumbent implementation over the recorded production trace, modulo a declared normalisation**. We give the criterion precisely, enumerate the sources of non-determinism that must be quotiented out and the technique for each, and analyse the two places the method can silently fail — classifier false negatives, and correlated defects between an implementation and the replacement derived from it. We are explicit throughout that the guarantee obtained is *behaviour preservation*, not correctness, and that it holds only over the observed input distribution.

Seven stages · triage → verdict = observationally equal ≠ divergent Est. reading time 15 min

1 · THE PROBLEM

Why legacy migration is hard, stated precisely

The difficulty is usually described as comprehension: the code is old, undocumented, and its authors have gone. That description is wrong, or at least it identifies the wrong bottleneck. Comprehension is expensive but tractable. The binding constraint is **acceptance** — given a candidate replacement, no one can decide whether it may be deployed.

This is a specific, named problem. A test consists of an input and an **oracle**: a mechanism that decides whether the observed output is acceptable. Weyuker's 1982 formulation of *non-testable programs* [1] covers exactly this case — programs for which no oracle exists, either because the correct output is unknown or because checking it is as hard as computing it. Barr et al. [2] survey the ways the oracle problem is attacked in practice; for a fifteen-year-old procedure with no specification, most of them are unavailable.

Note what follows. Generating a replacement is cheap and getting cheaper; deciding whether to accept one is neither. Any method that speeds up generation without addressing acceptance moves the bottleneck rather than removing it.

> **The pivot.** When modernising, you do not need to know what the code *should* do. You need to know what it *did*. Replace the unavailable specification oracle with a **pseudo-oracle**: the incumbent implementation itself, which is running in production and can be observed. Correctness is redefined from "satisfies the intent" to "agrees with the incumbent on observed inputs".
>
> This is the same move as differential testing [3] and back-to-back testing of N-version systems, applied in the one setting where a trustworthy reference implementation is guaranteed to exist — because it is the thing you are replacing.

---

2 · THE CRITERION

What the method actually decides

Making the criterion explicit is what separates this from "we ran it and it looked fine". Everything downstream — which procedures are eligible, which differences matter, when a migration is done — is a consequence of the definition below.

DEFINITION — OBSERVATIONAL EQUIVALENCE OVER A TRACE

Let `P` be the incumbent procedure and `P'` the replacement. A recorded **call** is a pair `c = (a, σ)` of arguments and the pre-state of the data it reads. Define the **observation**

```
O(P, c) = ⟨ returned rows, write-set, raised errors ⟩
```

where the write-set is the complete set of intended row mutations, captured inside a transaction that is rolled back. Let `N` be a **normalisation**: a total function quotienting out declared non-determinism (§5). For a recorded trace `T`, the replacement is accepted iff

```
∀ c ∈ T .  N(O(P, c)) = N(O(P', c))
```

Each `c` for which this fails is a **divergence** and is adjudicated by a human.

Three properties of this criterion are worth stating plainly, because each is a limitation that no amount of engineering removes.

WHAT THIS DOES NOT ESTABLISH

It is not program equivalence. Equivalence of arbitrary programs is undecidable; this is equality of normalised observations over a *finite sample*. It says nothing about `c ∉ T`.

It is not correctness. If `P` is wrong and `P'` reproduces the error, the criterion is satisfied. Preserving a defect is a pass, by construction and on purpose (§4, stage 06).

Its strength is inversely proportional to `N`. Every field added to the normalisation is a field on which nothing is checked. A growing `N` is the clearest early signal that a migration is drifting toward vacuity, and it should be tracked as a first-class metric (§8).

WHAT IT DOES ESTABLISH

For every call in the recorded trace, the replacement returns the same rows, writes the same rows, and raises the same errors as the code currently in production — up to explicitly declared and individually justified normalisations. Where `T` is drawn from real traffic, this covers the input distribution the system actually experiences, in the proportions it experiences it, including states no one would have thought to construct by hand.

That is a weaker claim than correctness and a far stronger one than any legacy estate typically has. It is also *sufficient* for a migration, whose contract is precisely "nothing observable changes".

3 · OVERVIEW

Seven stages — the code changes at stage six

Stages 01–05 change only what is *known* about the estate. The artefact itself is untouched until 06, and the incumbent continues serving production through 07.

Stage	Name	Output	Establishes
01	**Triage**	Every procedure classified by what evidence could ever establish equivalence	`eligibility`
02	**Spec**	Plain-language behaviour, invariants, and an open-questions register	`legibility`
03	**Oracle**	Trace `T` from recorded production calls, plus asserted invariants	`the pseudo-oracle`
04	**Baseline run**	Replay of `T` against `P` itself	`soundness of N`
05	**Diff classification**	Divergences partitioned into normalisable noise and real change	`tractability`
06	**Implement**	`P′` — the first point at which any code changes	`the candidate`
07	**Verdict run**	Replay of `T` against `P′`, rolled back	`acceptance`

4 · THE MECHANISM

01 · TRIAGE

Sort by what can be proved, not by what looks hard

The first pass does not read for meaning; it reads for **verifiability**. For each procedure it decides a single question: what class of evidence could, even in principle, establish that a replacement is observationally equivalent? The answer is a property of the code — decidable by static analysis over the parse tree, not a matter of judgement — and it determines the maximum autonomy any agent can ever be granted over that procedure.

> `pure read`
> Returns rows, writes nothing. Observation reduces to the result set; replay is side-effect-free and can be run against production directly.

> `deterministic write`
> Same inputs, same write-set. Requires transactional capture and rollback, but the criterion of §2 applies unmodified.

> `non-deterministic`
> Depends on the clock, generated identifiers, or unstable ordering. Not eligible until a seam exists that makes each such dependency injectable or normalisable (§5).

> `external effect`
> Reaches outside the transactional boundary — message queues, filesystem, network. Cannot be replayed without a side effect escaping, so cannot be verified this way at all.

This is the pass no human performs across ten thousand procedures, and it is the one that determines everything afterwards. Classification is cheap, mechanical, and total; it should never be sampled.

02 · SPEC

Make it legible before you make it different

An agent reads each procedure and records what it does: purpose, inputs, behaviour branch by branch, the invariants that appear to hold, the tables and columns touched — and, most valuably, an **open-questions register**: the behaviour it cannot distinguish as intentional from behaviour that merely looks like a mistake.

The register is not commentary. It is the input to stage 06's decision procedure, and later the reference against which stage 07's divergences are triaged: a divergence that corresponds to a logged open question is a different, and much cheaper, conversation than one that does not.

> **Precedent.** Morgan Stanley applied this at scale in 2025 — an internal tool read roughly nine million lines of legacy code and produced plain-English specifications, which the company estimated saved on the order of 280,000 developer hours. Note what was produced: not a rewrite, but the artefact that makes a rewrite reviewable.

---

03 · ORACLE

Constructing the pseudo-oracle

Every call in production is recorded: arguments, the returned rows, and the complete set of rows written. These recordings constitute `T`. Deduplicated by normalised input shape, a fortnight of traffic on a moderately used procedure typically yields on the order of 10^3 distinct cases from 10^4–10^5 calls.

The essential property is that `T` is *sampled from the true input distribution* rather than authored. Hand-written suites encode the author's model of the system, which is precisely the artefact that has been lost. Recorded traffic encodes the system's actual behaviour, including the states nobody would have thought to construct — and, importantly, in their real proportions, so that review effort lands where the volume is.

> **Invariants.** Alongside `T`, assert the properties that should hold on *every* run, not just recorded ones — e.g. `amount = round(base × rate, 2) + adjustment`, or "exactly one ledger row per (rep, period) after a run". These are metamorphic-style relations [5]: they extend coverage beyond `T` and are the only check that applies to inputs never observed.

> **The analogy, for non-specialists.** A bookkeeper of fifteen years is leaving; she computes commissions by rules that were never written down. You do not ask the new hire whether she can do it. You have her work in parallel for a month and compare the numbers. The parallel run *is* the oracle. You never needed the rules — you needed a reference and a comparison.

04 · BASELINE SHADOW RUN

Prove the harness before trusting the verdict

Before anything is replaced, `T` is replayed against `P` — the incumbent itself. The expected result is total agreement, which is exactly why the step is skipped, and exactly why it must not be.

Formally, it discharges the null case of the criterion: it establishes that `N(O(P, c)) = N(O(P, c))` holds *as executed by the harness*, which is not a tautology. It fails whenever replay does not faithfully reconstruct the pre-state, the clock or identifier seams are not properly pinned, ordering is not canonicalised, or the comparator itself is generating spurious differences.

> **Interpretation.** Baseline divergence rate is a measurement of the harness, not the code. Any non-zero rate is an upper bound on the precision of every subsequent verdict: if `P` cannot be shown equal to itself, no statement about `P′` carries information.

05 · DIFF CLASSIFICATION

Thousands of differences, twelve that matter

Replay produces a large volume of raw field-level differences, the overwhelming majority of which are artefacts of non-determinism rather than behaviour. Reduction proceeds in two stages, and the ordering is not incidental: **mechanical normalisation is applied first and exhaustively**, because every difference resolved by a rule is one not entrusted to a model. Only the residue is passed to a classifier, which labels each as normalisable noise — with a stated, auditable reason — or as a real behavioural change, with an account of what changed and its business consequence.

RAW DIFFERENCES	CLASSIFIED NOISE	TO A HUMAN
1,847	1,835	12
from one replay	each with a stated reason	real behavioural change

This step is what makes the method scale. Without it the bottleneck is not removed but relocated — from writing code to reading diffs, which is strictly worse, because diff review is less interesting and no faster.

THE METHOD'S MOST DANGEROUS FAILURE MODE

The classifier's **false negatives** — a real behavioural change labelled as noise — are the only errors in the entire method that are silent. A false positive costs a human five minutes. A false negative ships a behaviour change with a certificate saying it did not happen.

Three controls follow, and none is optional if the pipeline runs unattended:

(i) Hard exclusions. The classifier is structurally forbidden from suppressing a difference in any field named by a declared invariant or by a primary/foreign key. These route to a human unconditionally.

(ii) Machine-checkable reasons. A suppression citing "clock-dependent field" is verified against the declared clock-dependent field set rather than accepted as prose. Reasons that cannot be checked mechanically are themselves a review queue.

(iii) Audited sampling. A random sample of the noise bucket is reviewed by a human every run, and the measured false-negative rate is published as a running metric. An unaudited classifier has an unknown error rate, and an unknown error rate is not evidence.

06 · IMPLEMENT

Now, and only now, write the replacement

With a specification, a suite drawn from real traffic, asserted invariants, and a harness validated against itself, writing P′ is the least demanding step in the sequence. It is also the first at which any code changes.

> **The dominating rule: preserve behaviour, do not fix bugs.** The open-questions register will contain things that look wrong, and some of them are. Reproduce them anyway. A behavioural fix bundled into a migration destroys the interpretability of the comparison — a divergence no longer distinguishes "the new code is broken" from "the new code is better", and the whole apparatus of §2 collapses. Fixes ship afterwards, as their own change, with their own evidence.

CORRELATED FAILURE — THE ASSUMPTION THIS METHOD DOES NOT MAKE

Classical N-version programming assumed that independently developed implementations fail independently. Knight and Leveson's experiment [4] refuted that assumption: independently written versions failed on correlated inputs far more often than independence predicts.

Here the situation is worse and should be stated as such. P′ is derived *from* P — via the specification, and often by a model that read the original directly. Their errors are not merely correlated; shared error is the design intent. A shadow run therefore detects **divergence**, never **shared defect**. This is not a flaw in the method; it is the precise reason its guarantee is stated as behaviour preservation in §2 rather than correctness.

07 · VERDICT SHADOW RUN

The replacement runs dry, beside the real thing

`T` is replayed against `P'`. The customer sees only the incumbent's answer; the replacement executes alongside inside a transaction that is always rolled back, so both the returned rows and the full intended write-set are observable while **production is never mutated**. Industrial precedents for exactly this pattern are well established — GitHub's Scientist `[7]` for in-process refactoring under live traffic, Twitter's Diffy `[8]` for service-level response comparison.

Matches accumulate; divergences enter the adjudication queue. And here is the empirically interesting part: **divergences reaching a human are frequently not defects in the replacement.** They are behaviours the incumbent has exhibited for years that no one knew about — a recalculation silently discarding manually entered corrections, a status code excluded since 2011 for reasons no longer recorded.

> **Discovery is not licence.** The correct response to such a finding is to preserve the behaviour, log it, and raise the fix as a separate change with its own evidence. A system that silently corrects what it finds has quietly reintroduced the acceptance problem the method exists to solve. It surfaces; it does not decide.

5 · NORMALISATION

Sources of non-determinism, and the technique for each

The normalisation `N` is the method's central engineering artefact and its central liability. Each row below is a class of divergence that is not behavioural; each is neutralised by a different mechanism, with a different cost to the strength of the criterion.

Source	Technique	Cost to the criterion
`wall-clock reads`	Virtualise the clock; pin to the recorded timestamp of the call	None — exact under injection
`generated identifiers (sequence, identity, UUID)`	Seed deterministically, or canonicalise to order of first appearance and compare structurally	None if referential structure is compared rather than literal values

Source	Technique	Cost to the criterion
`unordered result sets`	Impose a total order on the projected tuple before comparison	Masks real ordering changes unless the query declares an ORDER BY contract
`floating-point and decimal tails`	Compare within a declared tolerance ε	Real — any divergence below ε is invisible; ε must be justified per column
`hash/set iteration order`	Canonicalise the collection before serialisation	None
`isolation-level artifacts (e.g. dirty reads under NOLOCK)`	None available — the observation is not a function of `(a, σ)`	Disqualifying — reclassify as non-deterministic (§4, stage 01)
`effects outside the transaction`	None available — rollback does not retract them	Disqualifying — external-effect class, not eligible

The last two rows are the reason triage precedes everything. A procedure whose observation is not a function of its recorded inputs and pre-state cannot be verified by replay at all, and no amount of normalisation changes that — it changes only whether the failure is visible.

6 · COVERAGE

What the recorded trace does not reach

Because `T` is a sample of observed traffic, its coverage is the coverage of production behaviour during the recording window — no more. Three gaps follow directly, and each has a specific mitigation.

Rare branches. Error handlers, compensation paths, and end-of-period logic may not execute at all in a two-week window. Mitigation: measure statement and branch coverage of `P` under replay of `T`, and treat uncovered branches as explicitly *unverified* rather than as passing. This number belongs in the migration's acceptance record.

Unreached states. A branch may be covered while the data states that make it interesting are not. Coverage is necessary, not sufficient; invariants (§4, stage 03) are the check that survives here, because they constrain all runs rather than recorded ones.

Distribution shift. `T` characterises past traffic. A migration verified against it is verified against the past; regulatory changes, new integrations, or seasonal load can move the distribution after cut-over. Mitigation: keep the shadow comparator running for a period *after* promotion, so divergence detection continues against live traffic rather than stopping at the verdict.

RECOMMENDED ACCEPTANCE RECORD

A migration is defensible when it publishes, per procedure: the size and window of `T`; baseline divergence rate; branch coverage under replay; the full contents of `N` with a per-entry justification; the classifier's audited false-negative rate; and every adjudicated divergence with its decision and rationale. Each of these is a number someone can disagree with, which is the point.

7 · LIMITS

Where it stops

A method that claims no limits should not be believed. These are not caveats appended for balance; each one determines work that must be staffed with people.

Knowledge never written down is unrecoverable. If a rule existed only in the head of someone who has left, an agent can read from the code what happens. It cannot read whether that was intended. The bug/feature distinction in fifteen-year-old code is not a question any model can settle, because the evidence required does not exist in the artefact.

Some changes are simultaneously irreversible and unverifiable. Schema migrations against a live core, and anything with accounting or regulatory consequence, fall outside the rollback-and-compare pattern entirely: the observation cannot be taken without committing it. These remain with people — not because the model is incapable, but because the loss function is asymmetric to a degree that dominates any efficiency argument.

Some code admits no oracle at all. Where the correct answer is *by definition* whatever the current implementation returns — a score accreted from a decade of rule changes — equivalence is provable and improvement is not. You can migrate it; the moment you want to change it, there is nothing to measure against. Plan for people there, and do not expect the situation to improve on its own.

And the criterion itself is a floor, not a ceiling. Everything in §2 concerns preservation. A system that only ever preserves accrues no interest on the investment; the value of the method is that it makes the estate *safe to change*, which is a precondition for improvement and not a substitute for it.

8 · INSTRUMENTATION

What to measure

Procedures migrated is a vanity metric: it measures throughput of a process whose value depends entirely on the trustworthiness of its verdicts. The following measure that.

`intervention rate`	Human adjudications per hundred procedures completed. The headline number: it is the direct measure of how much of the work the system can actually carry, and the only one that determines whether autonomy can safely widen.
`baseline divergence`	Divergences when replaying `P` against itself. Must be zero, or every downstream verdict inherits the noise as an error bar.
`|N| · normalisation surface`	Count of fields quotiented out, and its trend. Rising `|N|` means the criterion is weakening; this is the leading indicator of a migration drifting toward vacuous agreement.
`classifier FNR`	False-negative rate from audited sampling of the noise bucket. Unmeasured, the classifier's output is an assertion rather than evidence.
`replay branch coverage`	Fraction of `P`'s branches executed under `T`. Bounds the scope of the guarantee, and identifies exactly what remains unverified.
`post-promotion divergence`	Divergences detected by the comparator left running after cut-over. The empirical check on distribution shift (§6).

THE METRIC THAT ACTUALLY MATTERS

Not ~~procedures migrated~~. How often a human has to step in.

When that number falls, the boundary of what agents may do unsupervised moves outward, and debt repayment becomes a by-product rather than a programme. It is also the honest measure of whether any of this works — a system requiring constant correction is telling you its instructions are wrong, not that the model is weak.

FURTHER READING

[1] E. J. Weyuker. On testing non-testable programs. *The Computer Journal*, 25(4), 1982. — The original formulation of the oracle problem.

[2] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo. The oracle problem in software testing: a survey. *IEEE Transactions on Software Engineering*, 41(5), 2015. — Taxonomy of oracle strategies, including pseudo-oracles.

[3] W. M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1), 1998. — Comparison against a reference implementation as an oracle substitute.

[4] J. C. Knight, N. G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1), 1986. — Why correlated failure must be assumed, not hoped against.

[5] T. Y. Chen et al. Metamorphic testing: a review of challenges and opportunities. *ACM Computing Surveys*, 51(1), 2018. — Invariant-style relations that extend coverage beyond a recorded trace.

[6] R. O'Callahan, C. Jones, N. Froyd, K. Huey, A. Noll, N. Partush. Engineering record and replay for deployability. *USENIX ATC*, 2017. — Practical constraints on faithful record-and-replay.

[7] GitHub. Scientist: a library for carefully refactoring critical paths, 2016. — Industrial precedent for running a candidate beside production traffic.

[8] Twitter. Diffy: testing services without writing tests, 2015. — Response-level differential comparison of a candidate against an incumbent.